

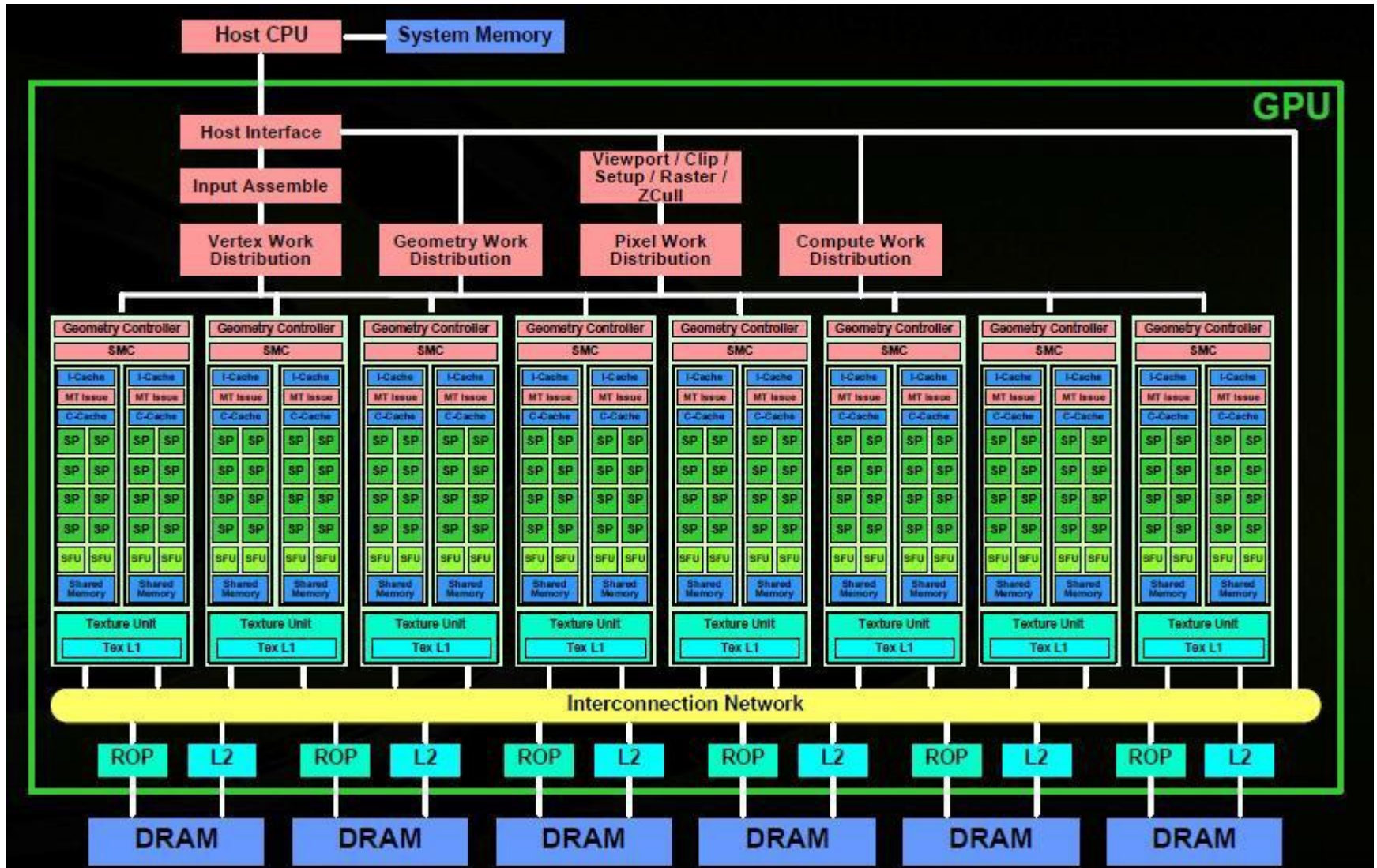
Parallel and distributed Computing

Lecture 14

Bazam-e-Jahan

Lecturer

The University of Faisalabad



- As we covered above, GPU parallel computing is the ability to perform several tasks at once. GPU Parallel computing enables GPUs to break complex problems into thousands or millions of separate tasks and work them out all at once instead of one-by-one like a CPU needs to.
- The GPU parallel computing ability is what makes GPUs so valuable. It is also what makes them flexible and allows them to be used in a wide range of applications, including graphics and video rendering.

What Are GPUs Used For?

- GPUs were invented to improve graphic rendering, but after some time, people started to realize that GPU parallel computing could also be used to accelerate a range of scientific applications, not just graphics rendering. This made them more flexible and programmable, enhancing their capabilities.
- Now, GPU parallel computing is used for a wide range of different applications and can be helpful for any device that needs to crunch a lot of data quickly.

**Gaming
Machine
Learning**

**Video Editing and Content
Creation**

- A GPU uses many lightweight processing cores, leverages data parallelism, and has high memory throughput. While the specific GPU architecture components vary by model, fundamentally most modern GPUs use single instruction multiple data (SIMD) stream architecture. To understand what that means -- and why it matters -- let's take a look at Flynn's Taxonomy.

What is Flynn's Taxonomy?

- Flynn's Taxonomy is a categorization of computer architectures by Stanford University's Michael J. Flynn. The basic idea behind Flynn's Taxonomy is simple: computations consist of 2 streams (data and instructional streams) that can be processed in sequence (1 stream at a time) or in parallel (multiple streams at once). Two data streams with two possible methods to process them leads to the 4 different categories in Flynn's Taxonomy.

Let's take a look at each.

Single Instruction Single Data

(SISD)

Single Instruction Multiple Data

(SIMD)

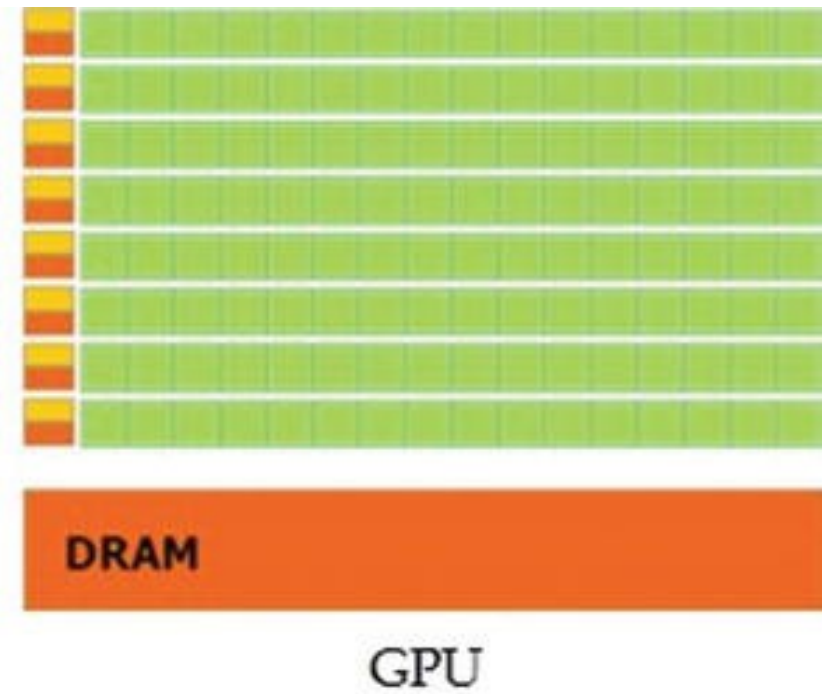
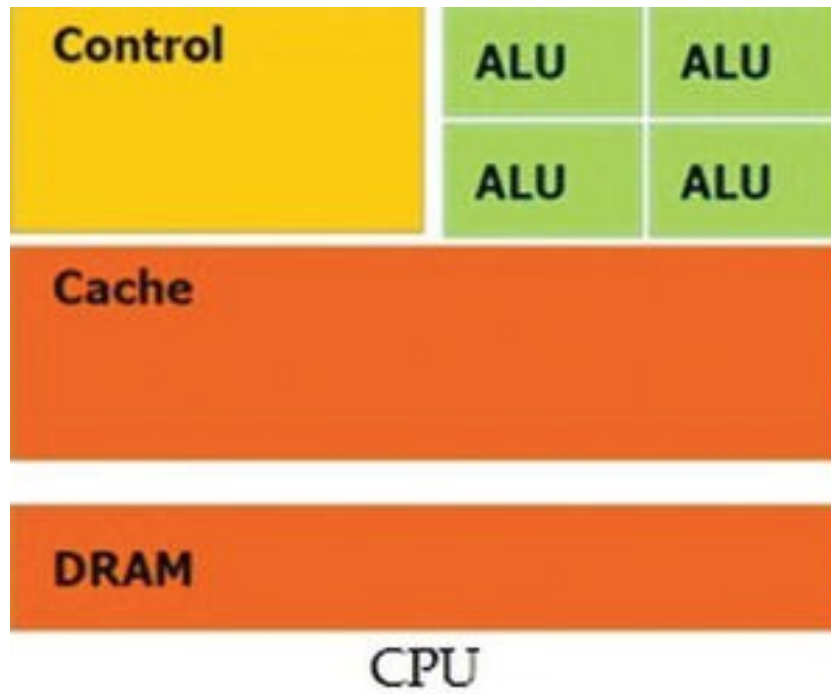
Multiple Instruction Single Data

(MISD)

What about SIMT?

- If you're familiar with GPUs, you've likely heard the term single instruction multiple threads (SIMT). So where does SIMT fit into Flynn's Taxonomy? SIMT can be viewed as an extension of SIMD. It adds multithreading to SIMD which improves efficiency as there is less instruction fetching overhead.

- GPU is a device that contains hundreds to thousands of arithmetic processing units (ALUs) with a same size. This makes GPU capable of running thousands of threads concurrently (able to do millions of similar calculations at the same time in parallel). These threads need to be independent of each other without synchronization issues to run concurrently. Parallelism of threads in a GPU is suitable for executing the same copy of a single program on different data [single program multiple data (SPMD)], i.e. data parallelism



- GPU architecture consists of two main components, global memory and streaming multiprocessors (SMs). The global memory is the main memory for the GPU and it is accessible by both GPU and CPU with high bandwidth. While SMs contain many simple cores that execute the parallel computations, the number of SMs in a device and the number of cores in SMs differ from one device to another.

- CUDA is a parallel computing platform and programming model invented by NVIDIA. It enables increases the computing performance by harnessing the power of the GPU devices. CUDA splits a problem into serial sections and parallel sections, serial sections are executed on the CPU as a host code and parallel sections are executed on the GPU by launching a kernel.

1. CUDA (Compute Unified Device Architecture):

1. A parallel computing platform and API developed by NVIDIA.
2. Allows developers to use C/C++ to write programs that run on NVIDIA GPUs.
3. Key Concepts:
 1. **Kernels:** Functions that run on the GPU.
 2. **Threads:** The smallest execution units.
 3. **Blocks:** Groups of threads that execute together on an SM.
 4. **Grids:** Collections of blocks.

2. OpenCL (Open Computing Language):

1. A framework for writing programs that execute across heterogeneous platforms (CPUs, GPUs, FPGAs).
2. Supported by both NVIDIA and AMD GPUs.
3. Key Concepts:
 - 1.**Kernels:** Functions that run on the device.
 - 2.**Work-Items:** Individual threads.
 - 3.**Work-Groups:** Groups of work-items.

3. HIP (Heterogeneous-Compute Interface for Portability):

1. A C++ runtime API developed by AMD for GPU programming.
2. Provides a CUDA-like interface for AMD GPUs.

Kindly read the book before and after the lecture, if some terms/ words are not covered in the lecture, if appear in exam it might not appear strange or out of course.